RESEARCH PAPER

Title:

LLMs vs Classical Algorithms: Efficiency and Reliability in Maze Pathfinding Tasks

Prepared by: deadline rusher

Suranchina Ainur, first-year student of IITU

For IRC ACADEMIXHUB96

1. **Abstract**

2. **Introduction**

3. **Literature Review**

4. **Methodology**

5. **Analytic / Results**

6. **Discussion**

7. **Conclusion**

8. **References + AI Usage Statement**

1. Abstract

This study investigates the effectiveness of large language models (LLMs) compared to classical search algorithms in solving structured pathfinding problems. Using randomly generated mazes, we evaluate Breadth-First Search (BFS), Depth-First Search (DFS), and A* alongside several LLMs, including LLaMA 3.1 8B, LLaMA 70B, and GPT-OSS 20B. The results show that classical algorithms consistently produce optimal solutions with negligible computation time, while LLMs demonstrate inconsistent performance, frequent failures, and significantly higher computational costs. The findings highlight the limitations of LLMs in structured reasoning tasks and raise questions about the efficiency of applying large-scale AI models to problems that can be solved deterministically

2. Introduction

Background

In recent years, artificial intelligence-particularly large language models (LLMs) - has gained significant attention due to its ability to solve a wide range of tasks. However, there is growing concern about the efficiency and practicality of using such models for problems that have well-established algorithmic solutions

Pathfinding in grid-based environments is a classical problem in computer science, traditionally solved using algorithms such as BFS and A*. These methods guarantee optimality and efficiency

At the same time, LLMs are increasingly applied to reasoning and planning tasks. This raises an important question:

Research Question

To what extent are large language models effective, reliable, and efficient compared to classical algorithms in solving structured pathfinding problems?

Problem Statement

While LLMs are powerful, they require substantial computational resources. There is a risk that they are being used in situations where simpler, more efficient methods already exist. This research explores whether LLMs are a justified solution for structured problems like maze solving

3. Literature Review

Classical pathfinding algorithms have been extensively studied and remain a fundamental part of computer science. Breadth-First Search (BFS) and Depth-First Search (DFS) are well-known graph traversal methods, while A* is a heuristic-based algorithm that guarantees optimal solutions under certain conditions. According to Introduction to Algorithms, BFS ensures the shortest path in unweighted graphs, whereas DFS explores depth-first and may produce suboptimal results. The A* algorithm, introduced by Peter E. Hart, Nils J. Nilsson, and Bertram Raphael, improves efficiency by incorporating heuristics, making it both complete and optimal in many scenarios

Recent advancements in artificial intelligence have introduced large language models (LLMs) as tools capable of performing reasoning-like tasks. Research by D. Wu, J. Zhang, and X. Huang suggests that techniques such as chain-of-thought prompting can enhance the reasoning capabilities of LLMs by encouraging step-by-step problem solving. However, despite these improvements, LLMs operate through probabilistic text generation rather than deterministic computation

This distinction highlights a key limitation: while classical algorithms are designed to systematically explore solution spaces and guarantee correctness, LLMs lack explicit mechanisms for state tracking and verification. As a result, their performance on structured tasks such as pathfinding remains uncertain. This gap motivates the present study, which empirically compares the reliability and efficiency of classical algorithms and LLMs in solving maze-based problems

4. Methodology

Approach

This study uses a **comparative experimental methodology**

All experiments were implemented in Python. The source code is available at:

https://github.com/Arafnel/LLMs-vs-algorithms-research-work

Setup

- 10 randomly generated mazes (size: 10×10; an example is shown below)

```
Original Maze:

S 0 1 1 1 1 1 1 1 1 1
1 0 0 0 1 0 0 0 0 0 1
1 1 0 0 1 0 1 0 1 0 1
1 0 1 0 1 0 1 0 1 0 1
1 0 1 0 1 1 1 0 1 0 1
1 0 1 0 1 0 0 0 0 0 1
1 0 1 0 1 0 1 1 1 0 1
1 0 1 0 0 0 1 0 0 0 1
1 0 1 1 1 1 0 0 1 1 1
1 0 0 0 0 0 0 0 0 0 1
1 1 1 1 1 1 1 1 1 0 G
```

- Algorithms:
  - BFS
  - DFS
  - A*
- LLMs:
  - LLaMA 3.1 8B
  - LLaMA 70B
  - GPT-OSS 20B

Prompt

```
messages = [
  {
    "role": "user",
    "content": f"Here is the maze\n{maze_str}\nYou are a pathfinding agent. Solve this {rows}x{cols} maze.\n\nGOAL: Move from 'S' to 'G'.\nRULES:\n- 'S' (Start) is at {start}\n- 'G' (Goal) is at {goal}\n- '0' are open cells.\n- '1' are walls. Do not walk into walls.\n- Output ONLY the sequence of moves (UP, DOWN, LEFT, RIGHT).\n- Think step by step about your current coordinates.\n\nINSTRUCTIONS:\nFind the path from {start} to {goal}.\nList EACH move on a new line with current coordinates.\nExample:\n(0,0) -> RIGHT -> (0,1)\n(0,1) -> DOWN -> (1,1)\n\nFinal list of moves: MOVE, MOVE, MOVE...\nReturn ONLY a list of moves at the end."
  },
  {
    "role": "assistant",
    "content": "I'm missing the start location. Could you provide the coordinates for **S** (row, column) in the same format as the goal? Once I know the start, I can calculate and list the moves."
  },
  {
    "role": "user",
    "content": f"S {' '.join(['1' if c == 'G' else str(c) for c in maze[0][1:]])}\n{maze_str[maze_str.find('\n')+1:]}\n\nYou are a pathfinding agent. Solve this {rows}x{cols} maze.\n\nGOAL: Move from 'S' to 'G'. RULES:\n\n   'S' (Start) is at {start}\n   'G' (Goal) is at {goal}\n   '0' are open cells.\n   '1' are walls. Do not walk into walls.\n   Output ONLY the sequence of moves (UP, DOWN, LEFT, RIGHT).\n   Think step by step about your current coordinates.\n\nINSTRUCTIONS: Find the path from {start} to {goal}. List EACH move on a new line with current coordinates. Example: (0,0) -> RIGHT -> (0,1) (0,1) -> DOWN -> (1,1)\n\nFinal list of moves: MOVE, MOVE, MOVE... Return ONLY a list of moves at
```

```
the end.\n"
  }
]
```

Metrics

- Path length
- Success rate
- Execution time
- Error types (invalid path, wall collision, incomplete solution)

5. Analysis / Results

5.1 Classical Algorithms

The figure below presents a visualization of a generated maze with overlaid paths produced by three classical search algorithms. In the grid, **S** denotes the start position, **G** the goal, **0** represents open cells, and **1** indicates walls. The paths found by the algorithms are marked as follows: **B** for Breadth-First Search (BFS), **D** for Depth-First Search (DFS), and **A** for A*.

The visualization illustrates that BFS and A* follow nearly identical routes, both achieving the shortest path from start to goal. In contrast, DFS deviates from the optimal path, exploring deeper branches and resulting in a longer route. The reported path lengths confirm this observation: BFS and A* both yield a path length of 29, while DFS produces a longer path of 31 steps

```
Combined Paths (BFS=B, DFS=D, A*=A):

S B 1 1 1 1 1 1 1 1 1
1 B B D 1 0 0 0 0 0 1
1 1 B B 1 0 1 0 1 0 1
1 0 1 B 1 0 1 0 1 0 1
1 0 1 B 1 1 1 0 1 0 1
1 0 1 B 1 B B B B B 1
1 0 1 B 1 B 1 1 1 B 1
1 0 1 B B B 1 B B B 1
1 0 1 1 1 1 D B 1 1 1
1 0 0 0 0 0 D B B B 1
1 1 1 1 1 1 1 1 1 B G

Lengths:
BFS: 29
DFS: 31
A* : 29
```

Results of all 10 cases in csv file:

```
maze_id,bfs_length,dfs_length,astar_length,bfs_time,dfs_time,astar_time
0,29,31,29,0.00012350082397460938,0.00012373924255371094,0.00014162063598632812
1,21,23,21,9.822845458984375e-05,0.00010085105895996094,8.130073547363281e-05
2,21,35,21,0.00010156631469726562,0.00010633468627929688,0.00015139579772949 22
3,21,21,21,8.702278137207031e-05,8.249282836914062e-05,0.00012540817260742188
4,29,37,29,0.00010848045349121094,9.942054748535156e-05,0.00016689300537109375
5,21,45,21,0.00010275840759277344,9.632110595703125e-05,6.67572021484375e-05
6,21,29,21,8.749961853027344e-05,0.00010776519775390625,7.128715515136719e-05
7,25,43,25,0.00010323524475097656,8.749961853027344e-05,9.846687316894531e-05
8,29,33,29,9.679794311523438e-05,0.00010013580322265625,0.00012946128845214844
9,21,49,21,9.512901306152344e-05,0.00010704994201660156,7.677078247070312e-05
```

- BFS and A* produced identical results in all cases
- Average path length: ~24.6
- DFS produced longer paths:
  - up to 49 steps vs 21 optimal
  - average overhead: ~10–20 steps

BFS and A* consistently produce optimal solutions, while DFS is less efficient

5.2 LLaMA 3.1 8B

Visualisation of process of thinking and example of the result:

```
Maze:
S 0 1 1 1 1 1 1 1 1 1
1 0 0 0 1 0 0 0 0 0 1
1 1 0 0 1 0 1 0 1 0 1
1 0 1 0 1 0 1 0 1 0 1
1 0 1 0 1 1 1 0 1 0 1
1 0 1 0 1 0 0 0 0 0 1
1 0 1 0 1 0 1 1 1 0 1
1 0 1 0 0 0 1 0 0 0 1
1 0 1 1 1 1 0 0 1 1 1
1 0 0 0 0 0 0 0 0 0 1
1 1 1 1 1 1 1 1 1 0 G

LLM raw output: To solve this maze, I will use a step-by-step approach. I will start at the 'S' (0, 0) and try to reach the 'G' (10, 10).

Current coordinates: (0, 0)
Current position: S

(0,0) -> RIGHT -> (0,1)
Current coordinates: (0, 1)
Current position: 0

(0,1) -> RIGHT -> (0,2)
Current coordinates: (0, 2)
Current position: 1 (Blocked by wall, cannot move right)

(0,1) -> DOWN -> (1,1)
Current coordinates: (1, 1)
Current position: 0

(1,1) -> RIGHT -> (1,2)
Current coordinates: (1, 2)
Current position: 0

(1,2) -> RIGHT -> (1,3)
Current coordinates: (1, 3)
Current position: 1 (Blocked by wall, cannot move right)

(1,2) -> DOWN -> (2,2)
Current coordinates: (2, 2)
Current position: 0

(2,2) -> DOWN -> (3,2)
Current coordinates: (3, 2)
Current position: 0

(3,2) -> DOWN -> (4,2)
Current coordinates: (4, 2)
Current position: 1 (Blocked by wall, cannot move down)

(3,2) -> RIGHT -> (4,2)
Current coordinates: (4, 2)
Current position: 1 (Blocked by wall, cannot move right)
```

```
(3,2) -> DOWN -> (4,2)
Current coordinates: (4, 2)
Current position: 1 (Blocked by wall, cannot move down)

(3,2) -> LEFT -> (3,1)
Current coordinates: (3, 1)
Current position

Parsed moves: []
LLM Path length: Invalid path
```

Result comparing to algorithms in 5 cases:

```
maze_id,bfs_length,dfs_length,astar_length,llm_length,llm_success,bfs_time,dfs_time,astar_time
0,29,31,29,0,False,0.00019407272338867188,0.00018286705017089844,0.00019741058349609375
1,21,23,21,0,False,0.00022125244140625,0.0002148151397705078,0.00017547607421875
2,21,35,21,0,False,0.00011324882507324219,0.00010991096496582031,0.00016427040100097656
3,21,21,21,0,False,0.0001652240753173828,0.00014853477478027344,0.00024127960205078125
4,29,37,29,0,False,0.0002944469451904297,0.00023698806762695312,0.0004336833953857422
```

- Success rate: 0% (0/5)
- Path length: always invalid
- Errors:
  - wall collisions
  - loops
  - incomplete paths

The model fails to solve even simple structured problems

5.3 LLaMA 70B

Visualisation of process of thinking and example of the result:

```
Maze:
S 0 1 1 1 1 1 1 1 1
1 0 0 0 1 0 0 0 0 1
1 1 0 0 1 0 1 0 1 0 1
1 0 1 0 1 0 1 0 1 0 1
1 0 1 0 1 1 1 0 1 0 1
1 0 1 0 1 0 0 0 0 0 1
1 0 1 0 1 0 1 1 1 0 1
1 0 1 0 0 0 1 0 0 0 1
1 0 1 1 1 1 0 0 1 1 1
1 0 0 0 0 0 0 0 0 0 1
1 1 1 1 1 1 1 1 1 0 G

LLM raw output: (0,0) -> RIGHT -> (0,1)
(0,1) -> RIGHT -> (0,2) is not possible, so (0,1) -> DOWN -> (1,1)
(1,1) -> DOWN -> (2,1)
(2,1) -> DOWN -> (3,1)
(3,1) -> DOWN -> (4,1)
```

```
(4,1) -> DOWN -> (5,1)
(5,1) -> DOWN -> (6,1)
(6,1) -> DOWN -> (7,1)
(7,1) -> DOWN -> (8,1)
(8,1) -> DOWN -> (9,1)
(9,1) -> DOWN -> (10,1)
(10,1) -> RIGHT -> (10,2)
(10,2) -> RIGHT -> (10,3)
(10,3) -> RIGHT -> (10,4)
(10,4) -> RIGHT -> (10,5)
(10,5) -> RIGHT -> (10,6)
(10,6) -> RIGHT -> (10,7)
(10,7) -> RIGHT -> (10,8)
(10,8) -> RIGHT -> (10,9)
(10,9) -> RIGHT -> (10,10)

RIGHT
DOWN
DOWN
DOWN
DOWN
DOWN
DOWN
DOWN
DOWN
DOWN
RIGHT
RIGHT
RIGHT
RIGHT
RIGHT
RIGHT
RIGHT
RIGHT
RIGHT
LLM Move DOWN hit a wall at (2, 1)

Parsed moves: ['RIGHT', 'DOWN', 'DOWN', 'DOWN', 'DOWN', 'DOWN', 'DOWN', 'DOWN', 'DOWN',
'DOWN', 'RIGHT', 'RIGHT', 'RIGHT', 'RIGHT', 'RIGHT', 'RIGHT', 'RIGHT', 'RIGHT', 'RIGHT']
LLM Path length: Invalid path
```

Result comparing to algorithms in 5 cases:

```
maze_id,bfs_length,dfs_length,astar_length,llm_length,llm_success,bfs_time,dfs_time,astar_time
0,29,31,29,0,False,0.00013327598571777344,0.00013184547424316406,0.00015354156494140625
1,21,23,21,0,False,0.00011014938354492188,0.00011396408081054688,8.535385131835938e-05
2,21,35,21,0,False,0.00019931793212890625,0.00019526481628417 97,0.0002968311309814453
3,21,21,21,0,False,0.00022268295288085938,0.0002014636993408203,0.00035572052001953125
4,29,37,29,0,False,0.00019121170043945312,0.00018334388732910156,0.0003058910369873047
```

Success rate: 0% (0/5)

- Behavior:
  - more structured reasoning
  - nearly correct paths
  - fails near completion

Increasing model size improves reasoning but does not guarantee correctness.

5.4 GPT-OSS 20B

Visualisation of process of thinking and example of the result:

```
From (0,0) to (0,1): RIGHT
(0,1) to (1,1): DOWN
(1,1) to (1,2): RIGHT
(1,2) to (1,3): RIGHT
(1,3) to (2,3): DOWN
(2,3) to (3,3): DOWN
(3,3) to (4,3): DOWN
(4,3) to (5,3): DOWN
(5,3) to (6,3): DOWN
(6,3) to (7,3): DOWN
(7,3) to (7,4): RIGHT
(7,4) to (7,5): RIGHT
(7,5) to (6,5): UP
(6,5) to (5,5): UP? Wait from 6,5 to 5,5 is UP. Actually earlier I wrote down but it's up. Let's double-check: 6,5 -> 5,5 is up. So we need UP.
(5,5) to (5,6): RIGHT
(5,6) to (5,7): RIGHT
(5,7) to (5,8): RIGHT
(5,8) to (5,9): RIGHT
(5,9) to (6,9): DOWN
(6,9) to (7,9): DOWN
(7,9) to (7,8): LEFT
(7,8) to (7,7): LEFT
(7,7) to (8,7): DOWN
(8,7) to (9,7): DOWN
(9,7) to (9,8): RIGHT
(9,8) to (9,9): RIGHT
(9,9) to (10,9): DOWN
(10,9) to (10,10): RIGHT

RIGHT, DOWN, RIGHT, RIGHT, DOWN, DOWN, DOWN, DOWN, DOWN, DOWN, RIGHT, RIGHT, UP, UP, RIGHT, RIGHT, RIGHT, RIGHT, DOWN, DOWN, LEFT, LEFT, DOWN, DOWN, RIGHT, RIGHT, DOWN, RIGHT
```

Result comparing to algorithms in 5 cases:

```
maze_id,bfs_length,dfs_length,astar_length,llm_length,llm_success,bfs_time,dfs_time,astar_time,llm_time
0,29,31,29,0,False,0.00015401840209960938,0.0001475811004638672,0.00016045570373535156,6.7719457149505615
1,21,23,21,21,True,0.00036263465881347656,0.0003058910369873047,0.00022149085998535156,3.4035451412200928
2,21,35,21,25,True,0.00035190582275390625,0.00029206275939941406,0.00046968460083007 81,6.612376928329468
3,21,21,21,21,True,0.00019550323486328125,0.00015592575073242188,0.00025200843811035156,29.92789101600647
4,29,37,29,0,False,0.00029277801513671875,0.0002682209014892578,0.0004630088806152344,17.75356388092041
```

Success rate: 60% (3/5)

- Path length:
  - sometimes optimal (equal to BFS)
  - sometimes longer

- Execution time:
    - 3 to 30 seconds per maze

Key Comparison

Method	Success Rate	Optimal	Time
BFS	100%	Yes	~0.0001 s
A*	100%	Yes	~0.0001 s
DFS	100%	No	~0.0001 s
LLaMA 8B	0%	No	1-3 s
LLaMA 70B	0%	No	1-3 s
GPT-OSS 20B	60%	Sometimes	3–30 s

LLMs are up to 100,000 times slower than classical algorithms while being less reliable

6. Discussion

The results highlight fundamental limitations of LLMs in structured environments:

1. Lack of deterministic reasoning

LLMs generate outputs probabilistically rather than following strict logic

2. Absence of state tracking

Models fail to maintain a consistent representation of the maze

3. Inefficiency

Even when successful, LLMs require significantly more time

Important Observation

GPT-OSS 20B demonstrates that:

- LLMs can approximate algorithmic behavior
- but only at a much higher computational cost

Broader Implication

This raises a critical concern:

The increasing reliance on large AI models may lead to inefficient use of computational resources when simpler solutions exist

7. Conclusion

This study demonstrates that classical algorithms significantly outperform LLMs in solving structured pathfinding problems.

- BFS and A* guarantee optimal solutions with negligible time
- LLMs show inconsistent performance and frequent failure
- Larger models improve performance but remain inefficient

Final Statement

While LLMs are powerful tools for many tasks, they are not a suitable replacement for classical algorithms in deterministic problem domains

8. References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms*. MIT press.
- Wu, D., Zhang, J., & Huang, X. (2023, July). Chain of thought prompting elicits knowledge augmentation. In *Findings of the Association for computational linguistics: aCL 2023* (pp. 6519-6534).
- Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE transactions on Systems Science and Cybernetics*, *4*(2), 100-107.

AI Usage Statement

AI tools were used for structuring the research paper, refining language, and assisting in data interpretation. All experimental design, coding, and analysis were conducted independently